

# PasteCraft Cross Sync Consultation Report

Architecture review • Cross sync feasibility • Data safety verification

Client: ezequielcasilla   Date: March 2026   Branch: pc1.0  
Repo: [github.com/ezequielcasillas1/PasteCraftio](https://github.com/ezequielcasillas1/PasteCraftio)

## What this report is

---

This report covers the full consultation review of the PasteCraft Edge extension, specifically the cross device sync architecture and the planned update that was in progress at time of review. No code was written during this engagement. The scope was to review the architecture, figure out why cross sync was not working, confirm whether the upcoming update would or would not erase existing user data, and give a clear list of what needs to be fixed and in what order. The review was done on branch `pc1.0` at the repository [github.com/ezequielcasillas1/PasteCraftio](https://github.com/ezequielcasillas1/PasteCraftio).

The short version is that the architecture is actually in good shape. The general approach, which is using Supabase as the central sync hub, using content hash deduplication to avoid duplicates, and offering a manual device to device import flow through a panel in the UI, is all reasonable and appropriate for what this extension is doing. The reason cross sync is not working has nothing to do with the architecture being wrong. It is two specific implementation bugs that are causing the entire feature to fail silently. Fix those two bugs and cross sync works. Everything else in this report is improvements, not blockers.

The update will not erase user data. This is confirmed, not assumed. The reasoning is laid out in full in the Data Safety section below.

## Storage layers and how data flows

---

PasteCraft uses four separate storage layers and it helps to understand all of them before getting into what is broken. Each layer has a different scope and different persistence characteristics.

| Layer                                             | What it holds — Persists across update                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|---------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>chrome.storage.local</code>                 | Clips, categories, notes, settings, the auth session, and the device ID. Survives update: Yes.                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <code>chrome.storage.sync</code>                  | The <code>accountUserId</code> which is used for cross browser identity. Survives update: Yes.                                                                                                                                                                                                                                                                                                                                                                                                                         |
| IndexedDB<br>( <code>pastecraft_local_v1</code> ) | A mirrored copy of clips, categories, and notes, each with a content hash. The migration uses a one time flag <code>pc_idb_migrated_v1</code> and does not re run destructively.. Survives update: Yes.                                                                                                                                                                                                                                                                                                                |
| Supabase (remote)                                 | All entities keyed by <code>user_id</code> plus entity specific ID. Row level security enforces <code>auth.uid()::text = user_id</code> on all tables. Tables include: <code>user_profiles</code> , <code>clips</code> , <code>archived_clips</code> , <code>categories</code> , <code>notes</code> , <code>note_versions</code> , <code>pastecraft_devices</code> , <code>device_sync_state</code> , <code>clipboard_history</code> , <code>settings</code> , <code>audit_log</code> .. Survives update: Server side. |

The data flow works like this. When something is written locally it goes to `chrome.storage.local` and IndexedDB at the same time. The cloud push uses `syncClipsToSupabase` with upserts keyed on (`user_id`, `clip_id`) and a `device_id` tag. The cloud pull uses `syncClipsFromSupabase` followed by `mergeClips`, which deduplicates using content hashes and resolves conflicts by timestamp, meaning the most recent version wins.

Device identity works like this: each install gets a random UUID stored in `pc_device_id_v1`. Devices register themselves in the `pastecraft_devices` table through `registerCurrentSyncDevice()`. When a user is authenticated, their identity comes from `auth.uid()`. For legacy flows there is also `chromeUserId` and `accountUserId`. The RLS policies on Supabase enforce `auth.uid()::text = user_id` on all tables.

There are two mechanisms that can both be active at the same time and they overlap. The first is bidirectional automatic sync through `performFullSync`, `mergeClips`, and realtime listeners. The second is the manual device to device import panel where a user opens the cross sync panel, sees other devices, and manually imports specific items. The flow for that goes through `listSyncDevices` then `getDeviceSyncMetadata` then `refreshSyncCandidates` then `getDeviceSyncData` then `importRemoteItem`. Because both mechanisms exist there are two ways to think about the design. Option A is to lean on auto sync so everything syncs automatically and the cross sync panel becomes more of a sync status view. Option B is to rely on manual import only, which is the stated intent, but that path requires the device filter bug below to be fixed first.

## Data safety: the update will not erase anything

This was reviewed carefully because it matters. Here is the specific evidence for why the update is safe.

| What was checked                                            | What was found                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>background.js</code> <code>onInstalled</code> handler | This handler only calls <code>createContextMenu()</code> . It does not read or write storage at all. There is no <code>clear</code> , no <code>reset</code> , nothing. Chrome and Edge both preserve <code>chrome.storage.local</code> , <code>chrome.storage.sync</code> , and <code>IndexedDB</code> across extension updates as a platform guarantee. The in memory state of the service worker is lost on update, which is expected and fine. |
| <code>handleUnsubscribe()</code> in <code>popup.js</code>   | This does call <code>chrome.storage.local.clear()</code> but only when the user explicitly chooses Delete all data in the UI. It is not triggered by an update event.                                                                                                                                                                                                                                                                             |
| IndexedDB migration                                         | The migration uses a one time flag called <code>pc_idb_migrated_v1</code> . Once that flag is set the migration does not run again. It is not destructive on re run anyway.                                                                                                                                                                                                                                                                       |
| Supabase schema changes                                     | Schema changes should be kept additive going forward. Do not drop columns or rename them in a way that breaks the existing extension code reading from those columns.                                                                                                                                                                                                                                                                             |

One thing worth adding as a precaution even though it is not strictly necessary: inside the `onInstalled` handler in `background.js`, add a check for `details.reason === 'update'` and log a message confirming that user data is being preserved. This does not change any behavior but it makes it much easier to debug in the future if someone ever raises a concern about data being lost. It is a one liner.

## Issues found

---

Six issues were found during the review. Two of them are the reason cross sync does not work at all. One is a data safety concern during interrupted syncs. The remaining three are improvements ranging from medium to low priority. Here is the full breakdown in priority order.

| Priority                  | Issues                                                                   | Why it matters                                                                                                         |
|---------------------------|--------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>P0 — Fix first</b>     | device_id filter bug and setUserContext RLS mismatch                     | Without these two fixes cross sync does not work at all. Device B cannot see Device A's items under any circumstances. |
| <b>P1 — Fix second</b>    | IndexedDB wipes data before rewriting                                    | If a sync is interrupted between the clear and the rewrite the local store is left completely empty.                   |
| <b>P2 — Improvements</b>  | No device display names and weak dupe detection for notes and categories | UX issue in the device panel plus a small edge case where the same note can be imported twice with a different ID.     |
| <b>P3 — House keeping</b> | Unused device_sync_state table                                           | The table exists in the schema but no extension code reads or writes it. No functional impact.                         |

P0  
CRITICAL

### **getDeviceSyncMetadata filters by device\_id and returns the wrong items**

Location: `extension/supabase-client.js` inside `getDeviceSyncMetadata()` and `getDeviceSyncData()` around lines 3609 to 3630

#### **What is happening**

Both functions filter their Supabase queries using `.eq('device_id', sourceDeviceId)`. The intent was probably to scope results to a specific device but the problem is that `device_id` records who uploaded the item, not where it was originally created. So if Device A creates a clip and that clip later gets synced through Device B, it gets stored in Supabase tagged with Device B's ID. When Device C later queries Device A, Device A shows nothing because the items are all tagged as Device B's. This is the primary reason Device B cannot see Device A's items at all. It has nothing to do with auth or Supabase permissions. The query is just filtering on the wrong thing.

#### **What to do about it**

Remove `.eq('device_id', sourceDeviceId)` from the clips, notes, and categories queries in both `getDeviceSyncMetadata()` and `getDeviceSyncData()`. The queries should filter by `user_id` and `is('deleted_at', null)` only. Client side deduplication already happens in `refreshSyncCandidates()` using content hashes and `origin_item_key`. That function should handle all the filtering. In terms of code change this is around six query lines. Small effort, highest possible impact.

P0  
CRITICAL

### **setUserContext calls an RPC that no RLS policy actually reads**

Location: `extension/supabase-client.js` inside `setUserContext()` around lines 1507 to 1542 and `db/supabase-schema.sql` RLS policies around lines 469 to 563

#### **What is happening**

The extension calls `rpc('set_config', { setting: 'app.current_user_id', value: userId })` through the `setUserContext()` function. The purpose was presumably to give Supabase a user context for RLS. The issue is that not a single RLS policy in the schema reads `app.current_user_id`. Every RLS policy uses `auth.uid()`. So for authenticated users this does not matter because `auth.uid()` works correctly. But for unauthenticated users or users going through the legacy `chromeUserId` flow, `auth.uid()` returns `NULL` and every single Supabase query returns zero rows silently. No errors are thrown. The queries just come back empty. This is almost certainly what was causing the Discord reports about RLS not working and users not being able to see their data.

#### **What to do about it**

Remove or replace `setUserContext()` with a no op and remove all calls to it. Cloud sync should only run when the user is authenticated via Supabase Auth, meaning `auth.uid()` returns a real value. `hasCloudSyncAccess()` already gates most of the cloud sync paths so this is mostly about removing dead code and making sure the remaining paths are covered. If a user is not signed in they should not be attempting Supabase queries at all.

#### P1 HIGH

### replaceFromAppItems clears the IndexedDB store before writing new data

Location: `extension/indexeddb-store.js` inside `replaceFromAppItems()` at line 84

#### What is happening

The function calls `store.clear()` and then inserts the new set of records. The problem with this approach is the gap between those two operations. If anything interrupts the process after the clear but before the writes finish, the store is left completely empty. In a browser extension context there are plenty of things that can cause this: the service worker can be killed by the browser, the user can close the browser, there can be a JS error midway through, or the device can lose power. Any of these scenarios leaves the user with no locally mirrored data. Because IndexedDB is the local mirror, this also means the sync state is lost until the next successful full sync from Supabase.

#### What to do about it

Replace the clear then refill pattern with a proper upsert in a single transaction. Write all new and updated records by primary key first. Then, in the same transaction, delete any records whose primary key is not present in the new set. This way nothing is deleted unless the new data has already been confirmed written. If the transaction fails or is interrupted nothing is lost because the old data was never touched.

#### P2 MEDIUM

### Devices always show as Device followed by their ID in the sync panel

Location: `extension/supabase-client.js` inside `registerCurrentSyncDevice()` around line 3556

#### What is happening

When a device registers itself in the `pastecraft_devices` table the upsert only sends `user_id`, `device_id`, and `last_seen_at`. The `display_name` column is never populated. So every device in the sync panel shows up as something like Device abc12345 and the user has no way to tell which device is which, especially if they have multiple installs across different machines.

#### What to do about it

On first registration set `display_name` from `navigator.userAgent` to produce something readable like Edge on Windows 11. Alternatively add a user facing setting where they can name the device themselves. Either way, pass the name in the upsert and make sure it is only set on INSERT so that a user chosen name does not get overwritten on subsequent registrations.

P2  
MEDIUM

### **Duplicate import is not fully prevented for notes and categories**

Location: `extension/popup.js` inside `importRemoteItem()` around lines 2694 to 2713

#### **What is happening**

Clips already have solid duplicate prevention. Before importing a clip the code checks the content hash and the `origin_item_key` to see if the item already exists locally. Notes only check by ID, which means a note with identical content but a different ID will be imported as a second copy. Categories only check by ID or name without accounting for icon, so a category with the same name but a different icon could also create a duplicate.

#### **What to do about it**

For notes, add a content hash check before importing. If a note with the same content hash already exists locally, skip the import. For categories, check by name and icon together rather than just ID or name alone. This brings notes and categories in line with how clips already handle deduplication.

P3 LOW

### **The `device_sync_state` table is defined in the schema but never used**

Location: `db/supabase-schema.sql` at line 152

#### **What is happening**

The `device_sync_state` table exists in the schema. It was presumably created for multi device reconciliation at some point. No extension code reads from it or writes to it. There is no functional impact from this.

#### **What to do about it**

Either wire it up as part of a future sync reconciliation feature or remove it from the schema to keep things clean. Not urgent at all.



# Step by step implementation plan

Here is the recommended order to work through the fixes. The P0 items should both be done together because they are both required before cross sync works at all. Test after each step before moving on.

1

## Apply both P0 fixes together

■ [extension/supabase-client.js](#)

Remove the `device_id` filter from `getDeviceSyncMetadata()` and `getDeviceSyncData()`. Then remove or no op `setUserContext()` and remove all calls to it. Confirm that cloud sync only runs when `auth.uid()` returns a real authenticated value. These two fixes address the main reason cross sync is completely broken.

2

## Test the P0 fixes with two devices or two browser profiles

■ [Two separate browser profiles or devices on the same account](#)

Sign in with the same Supabase account on both. Create a few clips on Device A. Open the cross sync panel on Device B. Device A should now appear in the device list. Selecting it should show Device A's clips as importable items. If this works the core bugs are confirmed fixed. If Device A still does not appear, check whether `registerCurrentSyncDevice()` ran on both devices and whether `listSyncDevices()` is returning the right results.

3

## Apply the P1 fix for IndexedDB

■ [extension/indexeddb-store.js](#)

Replace the `store.clear()` then put pattern in `replaceFromAppItems()` with upsert logic in a single transaction. To test this, create several clips, trigger a sync, and then interrupt it partway through by closing the extension or the browser. Reopen and verify all clips are still there. The old code would leave the store empty.

4

## Apply the P2 improvements

■ [extension/supabase-client.js](#) and [extension/popup.js](#)

Add device display names in `registerCurrentSyncDevice()`. Add content hash deduplication for notes in `importRemoteItem()`. Add name plus icon deduplication for categories in the same function. Retest the sync panel to make sure device names are showing up correctly and that importing the same note twice does not create a duplicate.

5

### Run the full testing checklist

■ [Full manual test pass before submitting to the Edge Store](#)

Go through every item in the testing checklist in this report before submitting anything to the Edge Store. Pay particular attention to the update safety test because that verifies the data preservation guarantee.

6

### Add the update safety log (optional but worth doing)

■ [extension/background.js](#)

Inside the `onInstalled` handler add a check for `details.reason === 'update'` and log that user data is being preserved. It is one line and it makes future debugging much easier if this ever comes up again.

# AI implementation prompt

---

This prompt can be dropped directly into any AI coding assistant to implement the fixes. It includes all the context the assistant needs without requiring you to explain the architecture from scratch. Copy and paste it exactly.

You are helping fix the cross device sync feature in PasteCraft, a browser extension that uses Supabase as its backend.

The codebase is in the pc1.0 branch of <https://github.com/ezequielcasillas1/PasteCraftio>.

Context:

PasteCraft stores clips, notes, and categories locally using `chrome.storage.local` and IndexedDB, and remotely using Supabase.

Each extension install has a unique `device_id` which is a UUID stored in `chrome.storage.local`.

Users authenticate through Supabase Auth using Google OAuth or email and password.

RLS policies enforce `auth.uid()::text = user_id` on all tables.

Cross device sync lets users manually import items from other devices through a panel in the UI.

Bugs to fix in order:

Bug 1: `extension/supabase-client.js` in `getDeviceSyncMetadata()` and `getDeviceSyncData()`  
Remove `.eq('device_id', sourceDeviceId)` from the clips, notes, and categories queries in both functions.

Query by `user_id` and `is('deleted_at', null)` only.

Let `refreshSyncCandidates()` handle client side filtering. It already does content hash and `origin_item_key` dedup.

Bug 2: `extension/supabase-client.js` in `setUserContext()`

Remove or no op the `rpc('set_config', { setting: 'app.current_user_id', value: userId })` call.

No RLS policy reads this setting. All RLS uses `auth.uid()`.  
Ensure cloud sync only runs when the user is authenticated. `hasCloudSyncAccess()`  
already enforces most paths.

Bug 3: `extension/indexeddb-store.js` in `replaceFromAppItems()`  
Replace the `store.clear()` then put pattern with a single transaction upsert.  
Put all new and updated records by primary key, then delete records whose primary key  
is not in the new set.  
Nothing should be deleted until the new data is confirmed written.

Bug 4: `extension/supabase-client.js` in `registerCurrentSyncDevice()`  
Set `display_name` on first registration using `navigator.userAgent` or a user customizable  
value.  
Pass it in the upsert and only set it on INSERT so a user chosen name is not  
overwritten.

#### Constraints:

Do not clear `chrome.storage.local` or IndexedDB on extension update under any  
circumstances.

Do not modify the `onInstalled` handler to clear user data.

All Supabase queries must work with the existing RLS which is `auth.uid()` based.

The cross sync panel should show items the user does not already have locally  
regardless of which device uploaded them.

Use `content_hash` and `origin_item_key` matching for duplicate prevention as already done  
in `refreshSyncCandidates()`.

Test with two devices on the same account: create clips on Device A, verify they appear  
as importable on Device B.

## Testing checklist before republishing

Run through all of these tests before submitting to the Edge Store. The P0 fixes should unblock the majority of them. The update safety test is particularly important because it confirms the data preservation guarantee.

| Test scenario           | What to verify                                                                                                                                                                                                                   |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Auth flow</b>        | Sign in with Google on Device A. Verify that clips sync to Supabase. Then sign in with the same account on Device B. Verify that Device A's clips are visible as importable in the cross sync panel.                             |
| <b>Cross sync panel</b> | On Device B, open the panel labeled "View Available Devices to Sync". Device A should appear as a listed device. Selecting Device A should show the clips, notes, and categories from Device A that are not already on Device B. |
| <b>Import</b>           | Import a clip from Device A to Device B. Verify it appears in Device B's local storage. Then import the same clip again and verify that no duplicate is created.                                                                 |
| <b>Bidirectional</b>    | Create a new clip on Device B. Open the cross sync panel on Device A. Device B should appear and the new clip should be listed as importable.                                                                                    |
| <b>Update safety</b>    | Install an older version of the extension and create several clips. Update to the new version. Verify all clips are still present both locally and in the cloud.                                                                 |
| <b>Offline</b>          | Create clips with no network connection. Go online. Verify the clips sync to Supabase without any data loss.                                                                                                                     |
| <b>Device limit</b>     | With ten or more devices or browser profiles connected to the same account, open the sync panel and verify that it shows a maximum of nine other devices, which is ten total including the current device.                       |

## File locations for reference

| File                         | What it does                                                                                                                                                                                                                                                                                                                                                       |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| extension/supabase-client.js | Everything Supabase related. Auth, sync queue, device registration, and all CRUD operations. This is where getDeviceId, getSyncUserId, setUserContext, hasCloudSyncAccess, registerCurrentSyncDevice, listSyncDevices, getDeviceSyncMetadata, getDeviceSyncData, syncClipsToSupabase, syncClipsFromSupabase, mergeClips, mergeCategories, and mergeNotes all live. |

|                                           |                                                                                                                                                                                                                                        |
|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>extension/popup.js</code>           | The device sync UI. Contains <code>refreshSyncDevices</code> , <code>refreshSyncCandidates</code> , and <code>importRemoteItem</code> .                                                                                                |
| <code>extension/indexeddb-store.js</code> | The local IndexedDB mirror. Handles <code>replaceFromApplItems</code> , <code>syncEntityFromLocalStorage</code> , and <code>ensureDeviceId</code> . Uses <code>content_hash</code> and <code>origin_item_key</code> for deduplication. |
| <code>extension/background.js</code>      | The service worker. The <code>onInstalled</code> handler here only calls <code>createContextMenus()</code> and does not touch storage. This is confirmed safe across extension updates.                                                |
| <code>extension/manifest.json</code>      | Manifest V3, version 3.0.6. Declares all permissions and host permissions for Supabase.                                                                                                                                                |
| <code>db/supabase-schema.sql</code>       | The full Supabase schema including all tables, RLS policies, <code>pastecraft_devices</code> , and <code>device_sync_state</code> .                                                                                                    |

## Summary

---

**The architecture is solid.** Supabase as the central sync hub, content hash deduplication, and the manual import panel are all the right approach for this kind of extension. The design does not need a rethink.

**Two bugs are breaking everything.** The `device_id` filter in `getDeviceSyncMetadata` and the `setUserContext` RLS mismatch are the reason cross sync does not work at all. These are the only things standing between the current state and a working cross sync feature.

**The update is safe.** Confirmed by reviewing the `onInstalled` handler, `handleUnsubscribe`, and the IndexedDB migration code. Nothing in the current codebase clears data on update. Chrome and Edge both guarantee storage persistence across extension updates at the platform level.

**Timeline.** The P0 and P1 fixes together should take one to two days to implement and test. The P2 improvements can follow. Once the full testing checklist passes the extension is ready to republish to the Edge Store.